1ACC0221 – BIG DATA

Sección: 18516

Program: Computer Science

Instructor: Carlos Adrian Alarcon Delgado

AI Repository Discovery

Data sources, dataset construction and exploratory analysis

Team Members:

Code	Full Name
u20211d574	Fabiana Nayeli Mallma Villanueva
u202323591	Luis Isaac Lazaro Machado

2026

Table of Contents

1. Project Proposal
2. Data Sources and Access
3. Dataset Design and Construction
4. Data Dictionary
5. Exploratory Analysis and Dataset Scale
6. Processed Dataset
7. EDA Analysis
8. Feature Matrix Construction
9. Dimensionality Reduction
10. Clustering and Validation
11. Recommendation and Offline Evaluation
12. Graph Analysis and Centrality
13. Conclusions and Limitations
14. Bibliography

> **Revision note.** This version updates the original report so that it matches the final project. Main changes: (1) the dataset is the 3,034-repository snapshot extracted from the GitHub API on 30 April 2026, not the 86-repository first sample; (2) Papers With Code and Hugging Face are listed as not used; (3) the recommendation results use one evaluation protocol for all systems, add a robustness test and an ablation, and withdraw the claim that SVD is the strongest system; (4) the graph rankings are compared against the full catalog and computed on the full main component; (5) course-week labels and the project's former internal name were removed. All figures were recomputed from the project files.

1. Project Proposal

This project is developed within the ecosystem of open-source scientific artificial intelligence tools. This domain was selected due to the accelerated and continuous growth of public repositories related to models, libraries, frameworks, and research-oriented tools. According to GitHub's Octoverse 2024 report, during 2024 developers created more than 70,000 new public and open-source generative AI projects, while total contributions to this type of project increased by nearly 60%. This growth reflects a highly dynamic, broad, and increasingly difficult environment to explore in a structured way.

In this context, the main problem is that the AI software ecosystem is growing in an explosive and disorganized manner. Every week, new GitHub repositories appear related to scientific papers, experimental implementations, or applied tools, but there is no structured system that allows users to discover, compare, and understand which repositories are truly influential beyond superficial metrics such as stars or forks. This creates difficulties in identifying similar repositories, recognizing thematic families, and prioritizing relevant tools within an increasingly large universe.

Based on this, the proposed system seeks to answer four specific questions:

- Which AI repositories exist within a specific domain, and which of them are similar to each other?
- To which thematic family or cluster does a given repository belong?
- What other tools should someone explore if they are already using a specific tool?
- Which repositories are structurally the most influential within the ecosystem, beyond their apparent popularity?

The dataset is suitable for the course because it combines free text in descriptions and topics, numerical variables such as stars and forks, temporal variables such as age and recent activity, and implicit relationships between repositories. This combination supports the subsequent stages of the project: representation, clustering, recommendation, and graph analysis.

2. Data Sources and Access

2.1. Source Inventory

Source: GitHub REST API
URL: https://api.github.com
License: Public API under GitHub Terms of Service.
Format: JSON via REST API (search endpoint), processed with PyGithub.
Current size: 3,034 repositories and 15 columns (10 collected fields and 5 derived features), extracted on 30 April 2026. The first iteration of the project used an 86-repository sample, kept in `data/raw/repos_raw.csv`.
Selection criteria: public repositories with more than 50 stars, found through search queries on AI research topics (machine learning, deep learning, NLP, computer vision, LLMs, among others). GitHub limits each search to 1,000 results, so several queries were combined and duplicates removed by `repo_id`.

Sources considered but not used. At proposal stage, Papers With Code and the Hugging Face Hub were considered to enrich each repository with its paper, benchmarks, models and datasets. They were not integrated: every column in the dataset comes from the GitHub API. They remain possible future work.

2.2. Ethics and Access Note

The data used in this project comes from public sources accessible through the official GitHub API, accessed with a personal token only for authentication and rate-limit control. No scraping of private profiles was performed, and no information restricted by third-party authentication was accessed. The extraction is limited to technical metadata from public repositories, such as descriptions, topics, primary language, stars, forks, and activity dates.

No real names, email addresses, private messages, or sensitive content from individual users were collected. Therefore, the risk related to personal data is minimal. The processed artifacts are used for academic purposes within the course.

3. Dataset Design and Construction

3.1. Schema

Column	Type	Description	Key
repo_id	integer	Unique identifier of the repository on GitHub	PK
nombre	string	Full name in `owner/repository` format	Unique
descripcion	string	Short description provided by the author	
lenguaje	string	Main programming language	
stars	integer	Number of accumulated stars	
forks	integer	Number of repository forks	
topics	list	Thematic tags declared by the author	
fecha_creacion	datetime	Repository creation date	
ultimo_commit	datetime	Date of the last recorded push	
url	string	Link to the repository	

3.2. Derived tables and joins

Table A	Table B	Join Field	Description	Status
repos	topics_expanded	repo_id	Topics normalized into one row per repository–topic pair (multi-label binarization), used for TF-IDF, SVD and the graph.	Implemented
repos	papers	nombre	Links each repository with its scientific paper.	Not implemented (future work)
repos	dependencies	repo_id	Links each repository with its declared libraries.	Not implemented (future work)

4. Data Dictionary

Observed values in the 3,034-repository snapshot.

Column	Type	Description	Range or observed values
repo_id	integer	Unique GitHub ID	265,995 – 1,223,170,290
nombre	string	`owner/repo`	Example: `tensorflow/tensorflow`
descripcion	string	Free text provided by the author	4 to 685 characters (median 79); “No description” if empty (3 repositories)
lenguaje	string	Main language	41 distinct values; Python (1,745), Jupyter Notebook (353), Unknown (287), C++ (189)
stars	integer	Superficial popularity	50 – 194,936 (median 2,174)
forks	integer	Number of times it was forked	0 – 75,296 (median 304)
topics	list	Author-defined tags	Between 1 and 20 per repository (median 8)
fecha_creacion	datetime	Creation date	31 Jul 2009 – 2026
ultimo_commit	datetime	Last recorded push	4 Dec 2017 – 30 Apr 2026
url	string	Direct link	Always `https://github.com/...`
dias_sin_commit	integer	Days from last push to the extraction date	0 – 3,069 (median 328)
antiguedad_dias	integer	Days from creation to the extraction date	2 – 6,116 (median 1,741)
num_topics	integer	Tag richness	1 – 20
ratio_forks_stars	float	forks / stars	0 – 2.29 (median 0.136)
stars_por_dia	float	stars / antiguedad_dias	0.02 – 2,420.5 (median 1.37)

5. Exploratory Analysis and Dataset Scale

The key figures of the current dataset are as follows:

Total repositories: 3,034
Total columns: 15
Missingness after cleaning: 0 across all columns (3 empty descriptions were filled with “No description” and 287 missing languages with “Unknown”)
Memory estimate: about 2.4 MB

Stars range: 50 to 194,936, with a median of 2,174. The distribution is strongly right-skewed: 549 repositories (18%) have 10,000 stars or more, while half have fewer than 2,200.

Unique languages: 41. Python is dominant with 1,745 repositories (57.5%), followed by Jupyter Notebook (353) and repositories with no language reported by GitHub (287; documentation or content not classified).

Repositories active in the last 30 days: 870 out of 3,034 (28.7%). 1,461 (48.2%) have not been pushed to for more than a year, which suggests that many research projects are published but not continuously maintained.

Average age: 1,889 days (approximately 5.2 years), so the dataset mixes established tools with recent projects; 2023 is the most frequent creation year.

The charts in section 7 show the distribution of stars (log scale), the ranking of programming languages, the days since the last commit, the activity categories and the creation year. They are generated by notebook 01 and stored in `reports/eda/`.

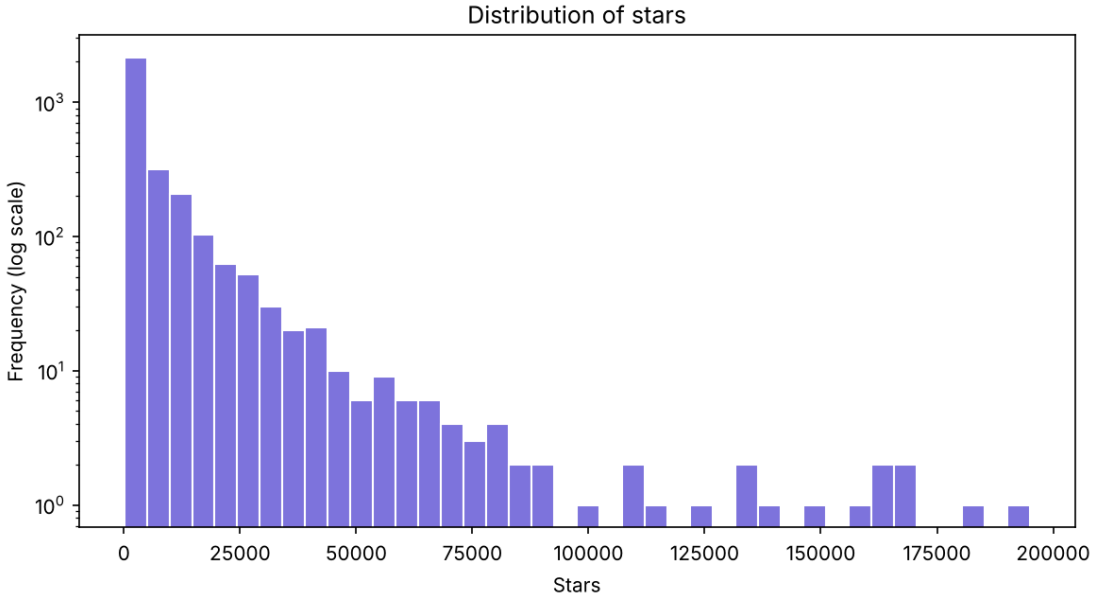
6. Processed Dataset

The processed dataset is generated through a pipeline organized in stages, implemented in `src/ingest.py`:

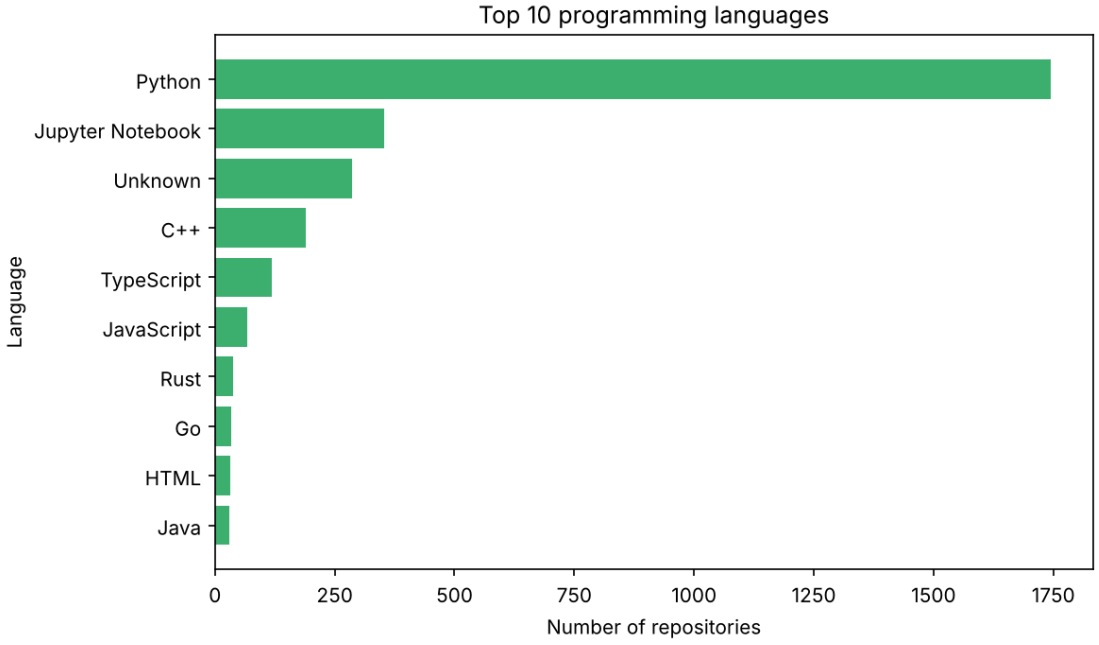
Step 1: the GitHub API is queried to collect the metadata of the repositories defined by the project's search queries.
Step 2: the extraction is stored as a raw table (`repos_raw.csv`). In the repository, `data/raw/repos_raw.csv` keeps the 86-repository sample from the first iteration; new extractions are written to `data/snapshots/<date>/`.
Step 3: the raw table is cleaned: empty descriptions and languages are imputed, dates are converted to UTC, and five features are derived: `dias_sin_commit`, `antiguedad_dias`, `num_topics`, `ratio_forks_stars` and `stars_por_dia`.
Step 4: the result is saved as the processed dataset `data/processed/repos_clean.csv` (the official 3,034-repository snapshot of 30 April 2026, used by every later stage and by the dashboard).

A new extraction can be launched from the project root with `python src/ingest.py`. By default it does not overwrite the official snapshot; `--promote` replaces it. Because stars and activity change daily, a new extraction will not reproduce exactly the numbers reported here.

7. EDA Analysis

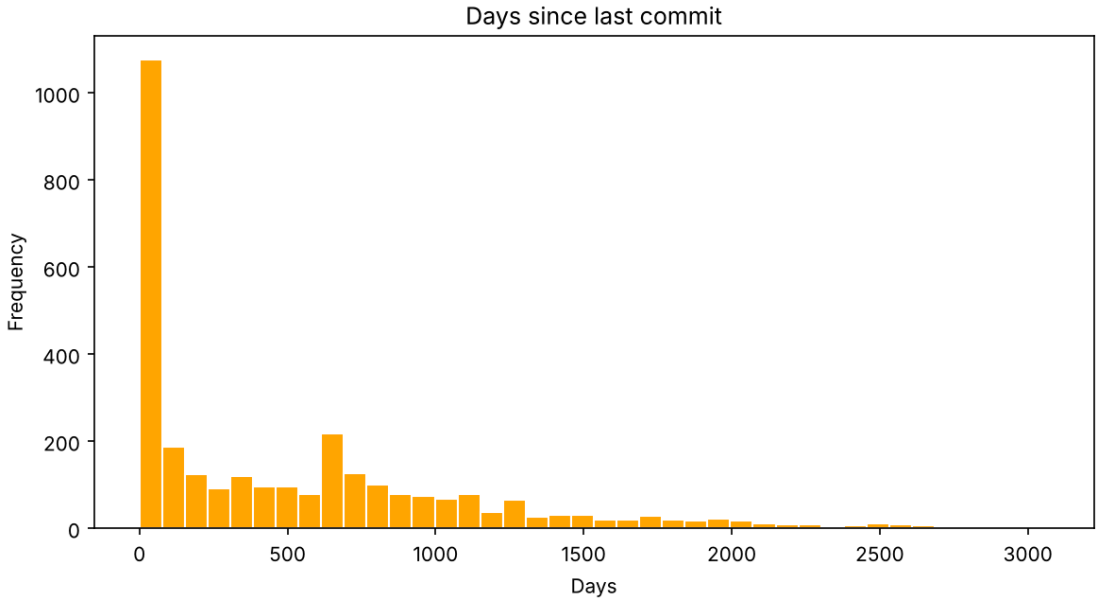


The distribution of stars shows a strong right-skewed pattern (log scale on the vertical axis). Most repositories have low or intermediate popularity, while a small group concentrates very high numbers of stars, up to almost 195,000. Visibility within the dataset is not distributed uniformly.

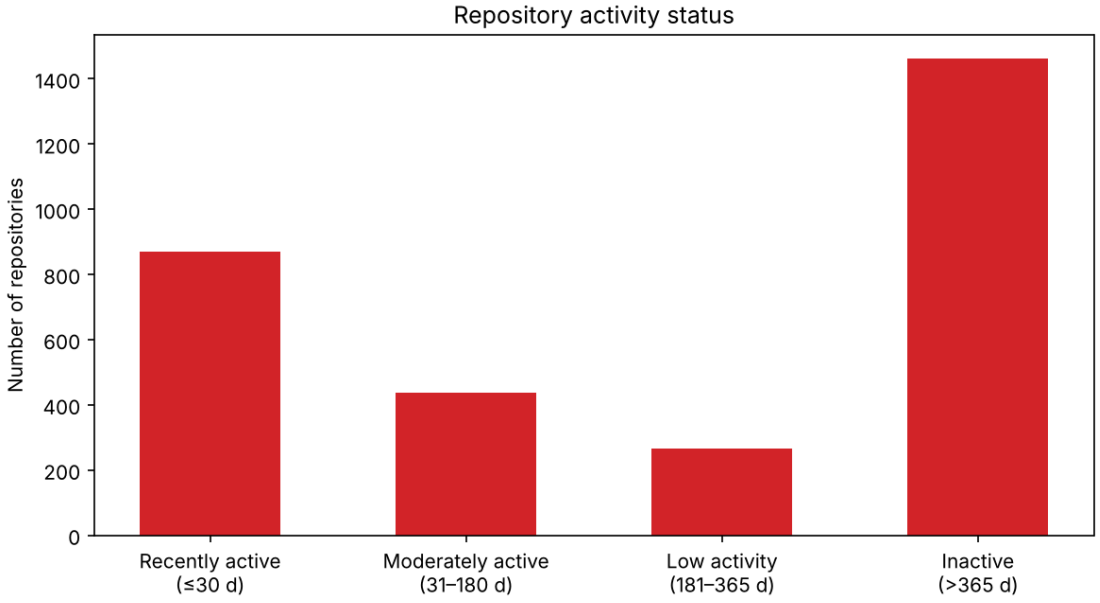


Python clearly predominates, followed by Jupyter Notebook and repositories with no declared language. This is consistent with the ecosystem analyzed, since Python is the most widely used

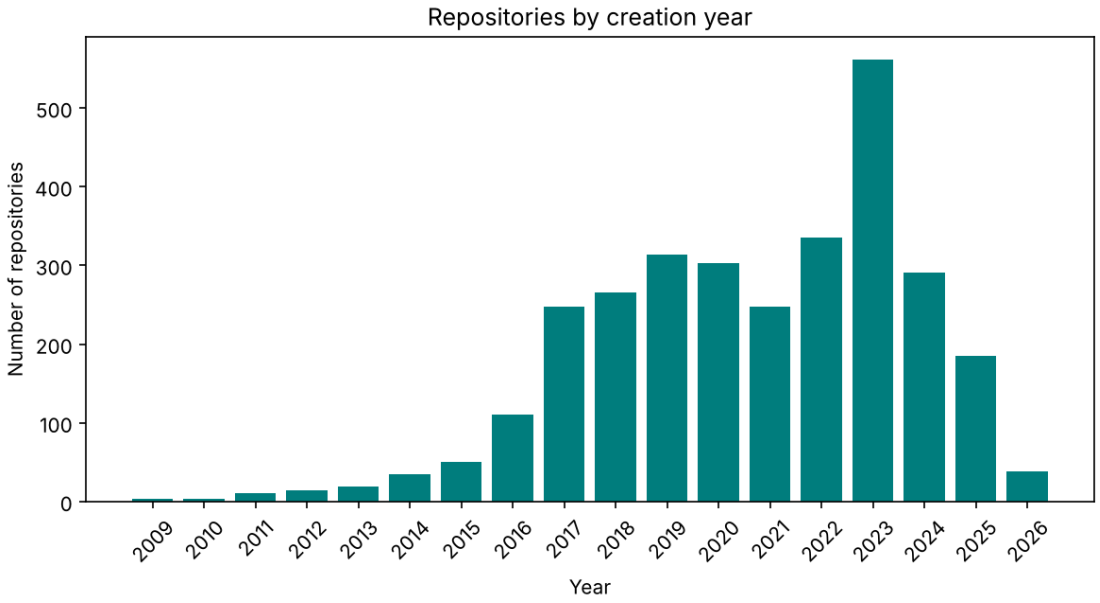
language in artificial intelligence and scientific software. C++ (189), TypeScript (118) and JavaScript (67) follow at a distance.



The distribution of days since the last commit shows a large group of recently updated repositories and a long tail of projects without updates. The dataset includes both actively maintained tools and repositories with limited or discontinued maintenance.



Activity categories use days since the last push: recently active (30 days or less, 870 repositories), moderately active (31–180 days, 436), low activity (181–365 days, 267) and inactive (more than 365 days, 1,461). The predominant category is inactive, so the collected ecosystem combines recognized projects with very different levels of recent maintenance.



Most repositories were created between 2017 and 2025, with 2023 as the peak year (562 repositories). The counts for 2026 are partial because the extraction took place on 30 April.

8. Feature Matrix Construction

8.1. Textual matrix (3,034 × 384)

The `descripcion` column of each repository was transformed into dense vectors with the pre-trained model `all-MiniLM-L6-v2`. It captures semantic similarity between descriptions rather than only word overlap, produces compact 384-dimensional vectors, and is cheap to run. This is useful because repository descriptions are short but state the purpose of the project.

8.2. Numerical matrix (3,034 × 7)

Seven variables describe popularity and activity: `stars`, `forks`, `dias_sin_commit`, `antiguedad_dias`, `num_topics`, `ratio_forks_stars` and `stars_por_dia`. The ratio measures active use (forks) relative to passive interest (stars); stars per day captures growth speed independently of age. All were standardized with `StandardScaler` because their ranges differ by orders of magnitude.

8.3. Language matrix (3,034 × 41)

One-hot encoding of the main language, one column per distinct language.

8.4. Topics matrix (3,034 × 30)

Multi-label binarization (`MultiLabelBinarizer`) of the 30 most frequent topics, since a repository can carry several topics at once. Limiting to 30 keeps the representative themes and avoids noise from rare topics. The matrix has a density of 8.91% (2.67 active topics per repository on average).

8.5. Combined matrix (3,034 × 128)

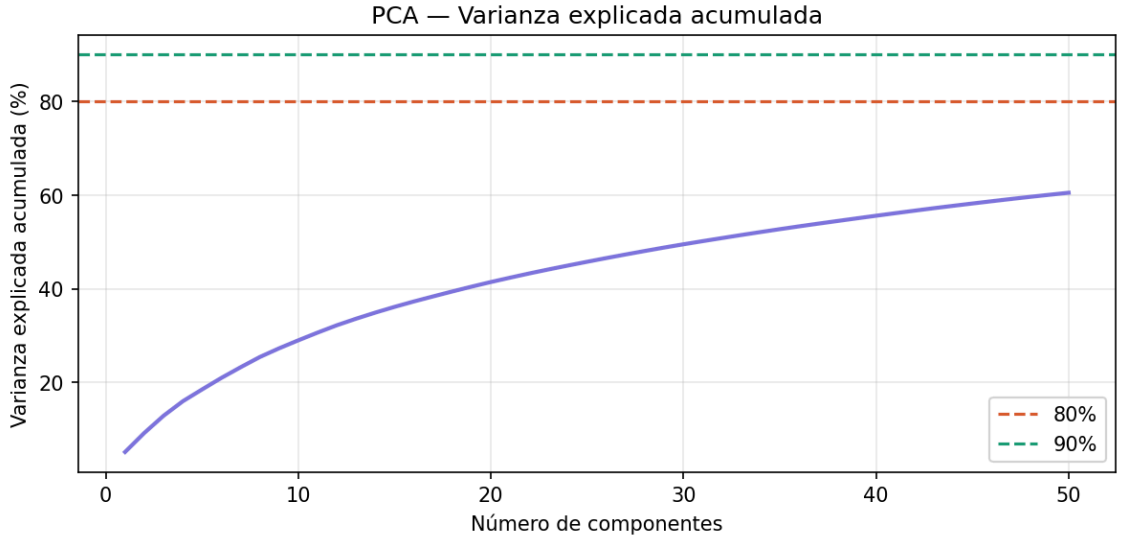
50 PCA components of the text embeddings + 7 standardized numerical variables + 41 language columns + 30 topic columns. It describes each repository from four perspectives (semantic content, quantitative behaviour, language and themes) and is the input of the clustering and recommendation stages.

9. Dimensionality Reduction

Dimensionality reduction was applied only to the text embeddings, the largest block (384 of the 462 raw dimensions before reduction).

9.1. PCA on the embeddings

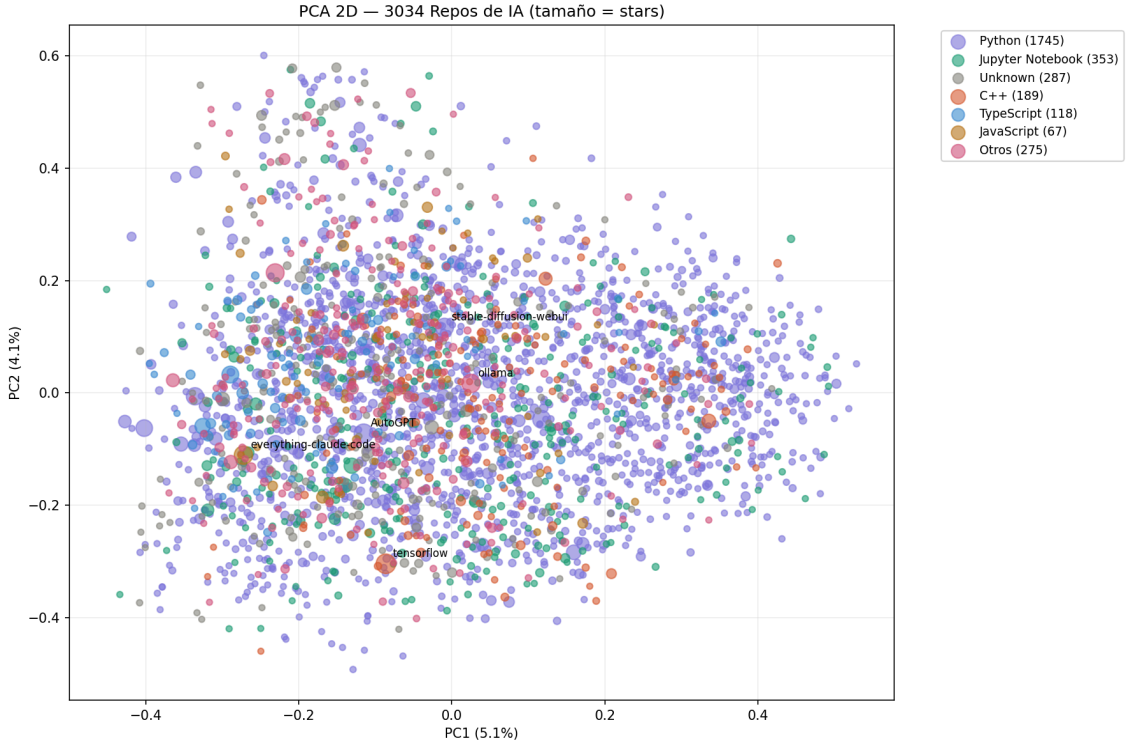
PCA was fitted on the 3,034 × 384 matrix with several numbers of components. Fifty components were retained: they cut the dimensionality from 384 to 50 while preserving about 60.5% of the cumulative variance. A high threshold such as 80% is not reached; the choice is a practical trade-off between computational cost and retained semantic information.



Representation	Dimensions	Explained variance	Use in the project
Original embeddings	384	100%	Full textual representation
PCA, 2 components	2	9.2%	2D visualization
PCA, 10 components	10	29.0%	Preliminary exploration
PCA, 20 components	20	41.4%	Information-retention comparison
PCA, 50 components	50	60.5%	Selected for the combined matrix
Combined feature matrix	128	n/a	Input of clustering and recommendation

The mean reconstruction error with 50 components is 0.001144 (average difference between the original embeddings and their reconstruction).

9.2. Two-component projection



The first two components capture only 9.2% of the variance (PC1 5.1%, PC2 4.1%), so the projection shows the general distribution of the repositories but no clearly separated groups. It is used for visualization only.

9.3. Interpretation

- Each description is a 384-dimensional vector with noticeable redundancy: 50 components keep about 60% of the variance.
- Clusters are not visible in 2D because two components explain a small share of the variance.
- The combined matrix mixes text, numerical metrics, language and topics, and is ready for clustering and recommendation.

10. Clustering and Validation

The repositories are segmented to identify patterns of popularity, activity, language and topics. The goal is not to group by text similarity alone but by overall behaviour, technology and thematic focus. The clustering representation combines the 7 standardized numerical variables, the 41 language columns and the 30 topic columns: **78 features** per repository.

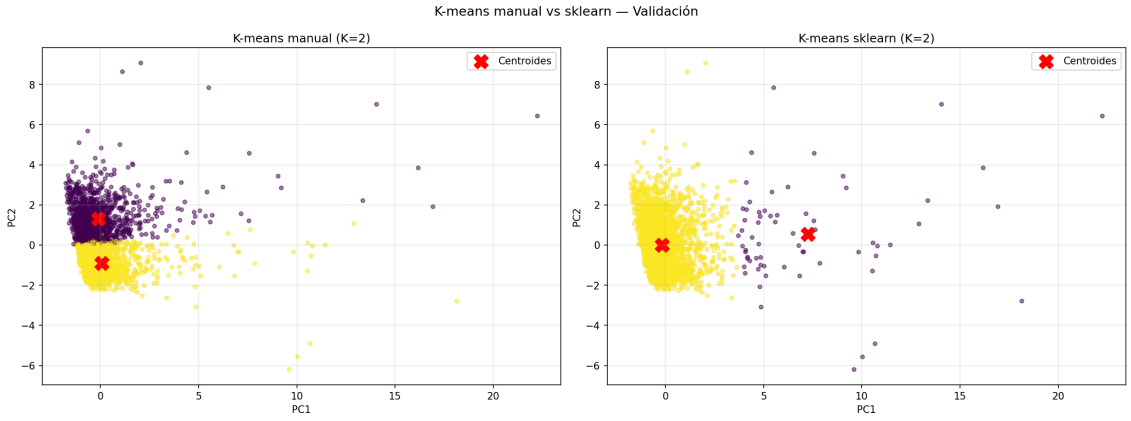
10.1. K-means from scratch

K-means was implemented with NumPy only, with four functions: `squared_distances` (squared Euclidean distance between points and centroids using broadcasting), `assign_clusters` (argmin over the distance matrix), `update_centroids` (mean of the assigned points) and `compute_inertia` (sum of squared distances to the assigned centroid). It stops when the centroid movement is below 1e-4 or the assignments no longer change.

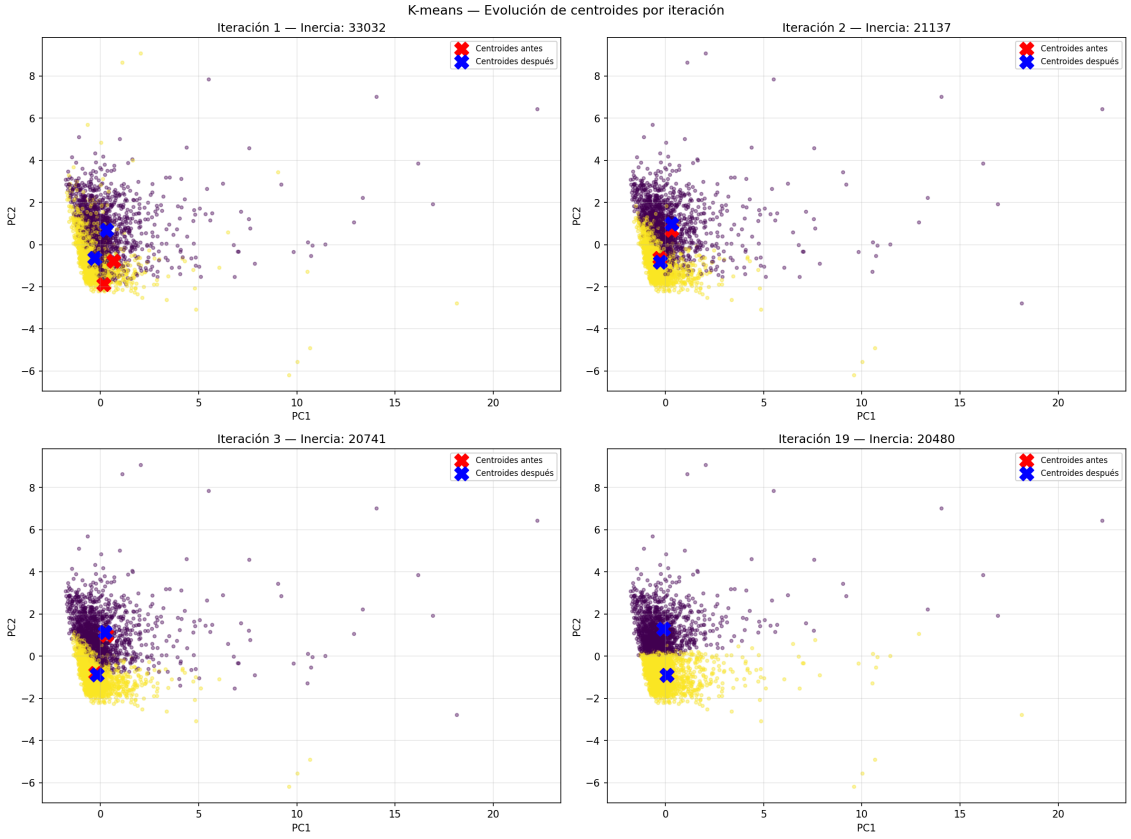
10.2. PCA before clustering

The 78 variables were reduced to 10 principal components (about 79.7% of the variance) because K-means relies on Euclidean distances and degrades in high dimensions with many binary columns. PC1 and PC2 explain 38.9% together and serve only as visual support: PCA preserves variance, it does not optimize cluster separation.

10.3. Validation against scikit-learn (K = 2)

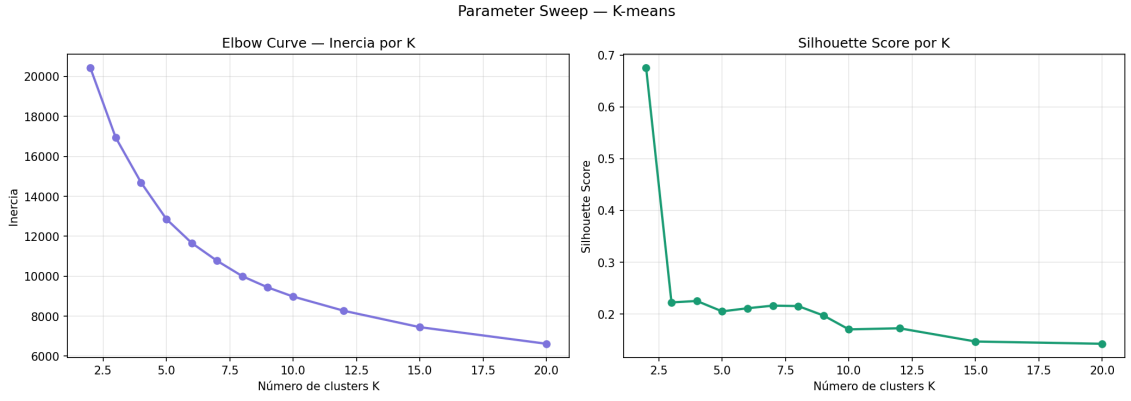


With K = 2 the manual implementation converged in 19 iterations with an inertia of 20,479, almost identical to scikit-learn's 20,427. The small difference comes from the initialization (scikit-learn uses K-means++), which confirms that the implementation is correct.



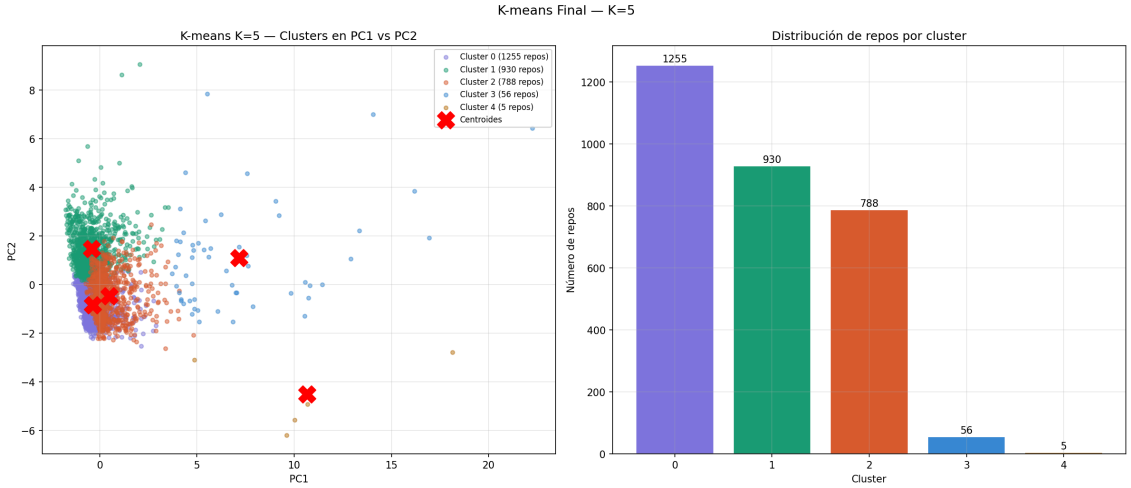
The first iterations show the centroids moving towards the mean of their assigned points; in the last iteration the centroids before and after the update overlap, which indicates convergence.

10.4. Choosing K



- K = 2 gives the highest Silhouette (0.675) but is too coarse for thematic analysis.
- From K = 3 the Silhouette stays around 0.22: boundaries are diffuse, as expected with overlapping thematic categories.
- Between K = 3 and K = 5 the reduction in inertia flattens. K = 5 was selected as a balance between interpretability and the elbow criterion.

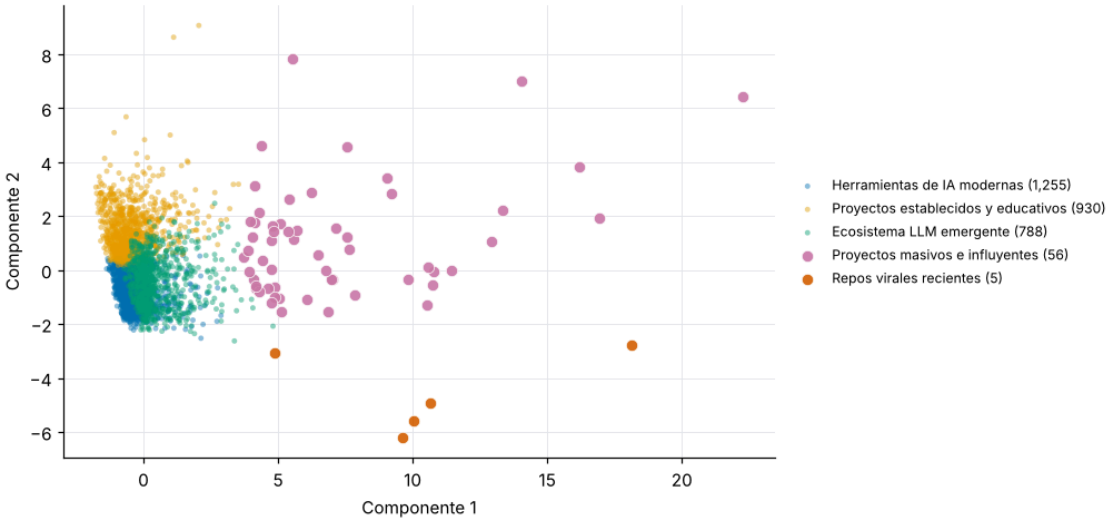
10.5. Final clustering (K = 5)



Clusters 0, 1 and 2 hold most repositories and overlap in the central area, so they should be read as exploratory segments rather than rigid partitions. Cluster 3 has 56 repositories and cluster 4 only 5; both are far from the main group. With K = 5 the manual inertia is 13,888 against 12,845 for scikit-learn, a larger gap than with K = 2 because the initialization matters more as K grows. The final clustering (silhouette 0.205) uses the scikit-learn result.

10.6. Cluster profiles

Cluster	Repos	Mean stars	Mean age (days)	Days since last push	Stars/day	Examples
0 Modern AI tools	1,255	4,070	1,170	336	6.3	whisper.cpp, GFPGAN, jan, context7
1 Established and educational projects	930	4,326	2,933	1,055	1.4	BERT, google-research
2 Emerging LLM ecosystem	788	8,409	1,798	228	8.9	unsloth, llama_index, mem0
3 Massive and influential projects	56	84,146	2,113	83	78.0	TensorFlow, Auto-GPT, Ollama, Stable Diffusion WebUI
4 Recently viral repositories	5	57,471	33	3	1,829	everything-claude-code, caveman, mempalace



Clusters differ mainly by popularity, age, recent activity and growth speed. Python is the main language of almost every group, so language alone does not explain the segmentation. Cluster 4 has only 5 repositories and must be interpreted with caution: very young projects (about a month old) with explosive growth.

10.7. DBSCAN as comparison

A sweep over `eps` and `min_samples` was run. With eps = 0.50 more than 79% of the repositories were noise; larger values merged almost everything into one cluster. The balanced setting, eps = 1.25 and min_samples = 5, found 2 clusters (2,771 and 5 repositories) and 258 noise points (8.5%), with Silhouette 0.258 excluding noise.

Model	Parameters	Clusters	Silhouette	Inertia	Distribution
K-means	K = 5	5	0.205	12,845.5	1,255 / 930 / 788 / 56 / 5
DBSCAN	eps 1.25, min_samples 5	2	0.258	n/a	noise 258; 2,771 / 5

DBSCAN's slightly higher Silhouette does not mean a more useful segmentation: it put almost the whole dataset in one cluster. K-means was kept as the main algorithm because it produces interpretable profiles.

10.8. Failure analysis and data-quality check

- The K-means Silhouette (0.205) is moderate to low, and the distribution is unbalanced (two clusters with 56 and 5 repositories).
- **Cause of the imbalance (tested).** Stars, forks and stars per day have very heavy tails and enter PCA without a log transform. Re-running the same pipeline with those three variables on a log scale gives a smallest cluster of 437 repositories (against 5), Silhouette 0.181 and a slightly higher recommendation precision (0.784 against 0.773). The

published clustering was kept for consistency with the rest of the report; the log version is a candidate for a second iteration.

- **Effect of the cleaning (tested).** Dropping incomplete rows instead of imputing them would lose 289 repositories (9.5%) and slightly lower the hybrid Precision@10 (0.755 against 0.773, approximate comparison because the catalog differs). Imputing empty descriptions and languages keeps the data and does not hurt precision.

11. Recommendation and Offline Evaluation

11.1. Task framing

System type	Applies?	Justification
Recommendation	Yes	The output is a ranked list of similar repositories for a query repository.
Ranking	Yes	Candidates are ordered by score (popularity, TF-IDF, SVD or hybrid).
Prediction	No	There is no supervised target variable.
Collaborative filtering	No	There are no users, so there is no user–repository interaction matrix.
Segmentation feeding ranking	Yes	The PCA space and the K-means clusters are additional proximity signals of the hybrid score.

Classic collaborative filtering needs a user–item matrix, and building one would require synthetic data. Instead, a co-occurrence proxy was used: a binary repository–topic matrix (3,034 × 30), to which truncated SVD (k = 10) was applied. The ten latent factors capture 58.42% of the variance; the first (12.21%) represents the dominant deep-learning / machine-learning axis. This applies matrix factorization to the only matrix structure available, without claiming behavioural collaborative filtering.

11.2. Systems compared

- **Popularity baseline:** recommends the most starred repositories regardless of the query. It fails by construction: the most starred repositories (tensorflow, pytorch, transformers) belong to different areas.
- **Content-based (description only):** TF-IDF on the description (5,000 terms) with cosine similarity.
- **Content-based baseline:** TF-IDF on description + topics (weighted ×3) + language.
- **SVD:** truncated SVD (k = 10) on the repository–topic matrix.
- **Hybrid:** weighted sum of six signals, `score = 0.35·content + 0.20·PCA similarity + 0.15·same cluster + 0.12·popularity + 0.10·activity + 0.08·growth`. Content has the largest weight as the most interpretable signal; popularity, activity and growth are secondary so that recent, maintained projects are favoured. PCA similarity and the cluster bonus come from the clustering stage.

11.3. Evaluation protocol

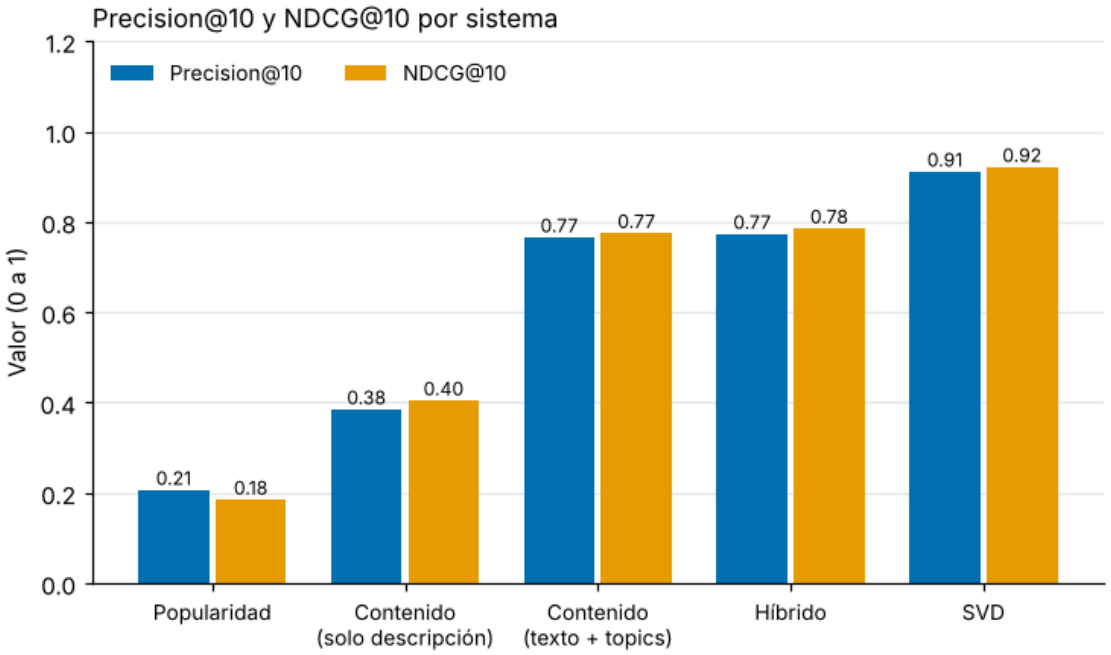
Offline evaluation needs no real users. Metrics are computed at K = 10: Precision@10 (fraction of the 10 recommendations that are relevant), Recall@10, MAP@10 and NDCG@10 (a relevant item at position 1 is worth more than at position 10), plus catalog coverage. The candidate pool is the whole catalog (3,034 repositories) without the query.

Relevance is a proxy: a repository is relevant if it shares enough of the 100 most frequent topics with the query (at least 2 if the query has three or more of those topics, at least 1 otherwise; the threshold avoids inflation from generic topics such as `python`). 500 queries were sampled with seed 42 from the 3,033 evaluable repositories, and every system is evaluated with exactly the same queries and rule. The dashboard recommends with the same code that is evaluated here, and tests check the published metrics.

An earlier iteration evaluated 50 random queries with a 30-topic relevance and reported Precision@10 of 0.108 (popularity), 0.274 (TF-IDF) and 0.680 (SVD). Those figures used a different protocol for each system family, are not comparable with the table below and are superseded by it.

11.4. Results

System	Precision@10	Recall@10	MAP@10	NDCG@10	Coverage@10
Popularity baseline	0.206	0.006	0.093	0.185	0.004
Content-based (description only)	0.384	0.016	0.272	0.404	0.646
Content-based baseline	0.765	0.037	0.680	0.775	0.731
Hybrid PCA + K-means	0.773	0.035	0.696	0.785	0.622
SVD (repository–topic matrix)	0.911	0.040	0.901	0.921	0.668

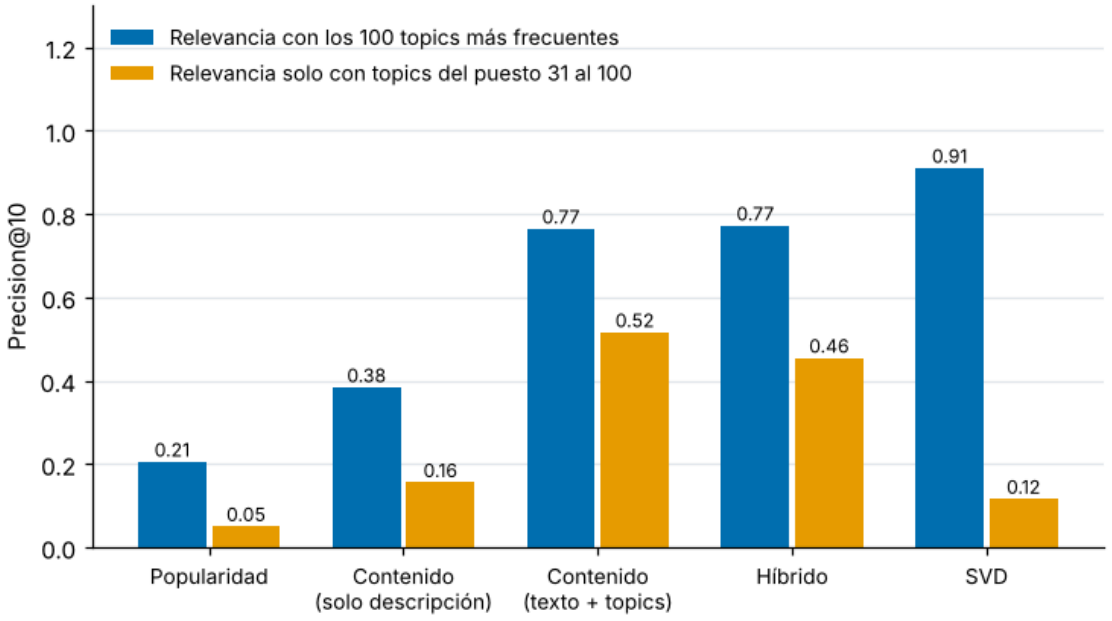


Recall@10 is low for every system because there are, on average, 367 relevant candidates per query and ten slots can never cover them: it is a structural limit, not a flaw.

11.5. SVD's lead is circular

SVD is built from the same topic vocabulary that defines relevance, so it is evaluated with the signal it was trained on. A robustness test redefines relevance using only topics ranked 31–100, which SVD (30 topics) never sees:

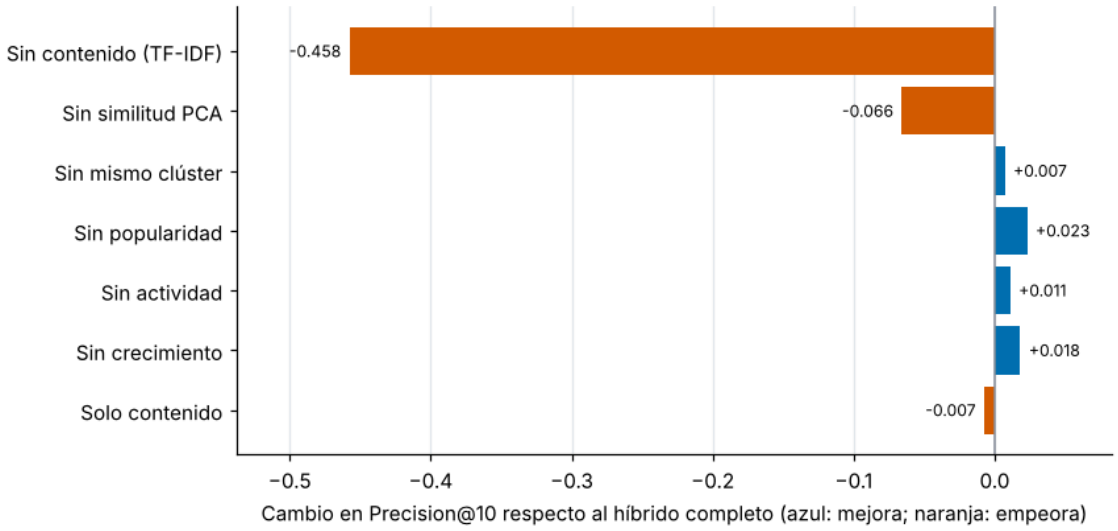
System	Precision@10, top-100 topics	Precision@10, topics 31–100
Popularity baseline	0.206	0.051
Content-based (description only)	0.384	0.156
Content-based baseline	0.765	0.516
Hybrid PCA + K-means	0.773	0.455
SVD	0.911	0.117



SVD collapses from 0.911 to 0.117, below the description-only TF-IDF, so its first place in the main table should not be read as the strongest system; it shows that topics are a strong signal when relevance is defined with topics. With relevance SVD cannot see, plain content-based is the best system and the hybrid is second. The earlier claims that "SVD is the strongest result" and that "topics are a more reliable signal than text" were drawn from the circular comparison and are withdrawn.

11.6. What each signal contributes

Variant	Precision@10	Δ Precision@10
Full hybrid	0.773	0
Without content (TF-IDF)	0.315	−0.458
Without PCA similarity	0.707	−0.066
Without same-cluster bonus	0.780	+0.007
Without popularity	0.796	+0.023
Without activity	0.784	+0.011
Without growth	0.791	+0.018
Content only	0.765	−0.007



- Textual content carries the system: removing it costs 45.8 points. PCA similarity adds 6.6 points.
- The hybrid beats plain content by only 0.7 points under the main protocol and is below it in the robustness test (0.455 against 0.516).
- Popularity, activity and growth slightly *lower* topical precision, as expected, because they push towards popular and recently maintained repositories. They are a product decision (recommend maintained, visible tools) and not an accuracy gain; the cluster bonus also has no measurable effect on precision.

11.7. Error analysis

Strong cases. Queries such as `keras-team/keras`, `ageitgey/face_recognition` and `JaidedAI/EasyOCR` reach Precision@10 = NDCG@10 = 1.0. They belong to broad, popular domains with 450–950 relevant repositories, so the top 10 is easily filled; several of them also have generic topics, which makes the proxy easy to satisfy.

Failure cases (Precision@10 = 0): `EutropicAI/Final2x`, `bytedance/SALMONN`, `ruvnet/RuVector`, `yazdotai/graph-networks` and `fangvv/LassoADMM`. Automatic diagnosis of the recommendations shows two patterns: (a) *limits of the proxy*: the recommended repositories share topics with the query that are outside the 100-topic evaluation vocabulary, so a recommendation that looks reasonable counts as an error; and (b) *popularity bias*: for `LassoADMM` the recommended repositories are much more popular than the query. Niche queries with very few relevant candidates (RuVector: 7, graph-networks: 17) are also hard by construction (metadata sparsity). An earlier version of this report attributed these failures to the PCA and cluster signals "pulling the ranking towards structurally similar repositories"; the diagnosis does not support that explanation.

11.8. Limitations of the evaluation

- Relevance is a topic-overlap proxy, not real clicks or ratings: the system is measured against a reproducible rule, not developer satisfaction.
- Cold start is not handled explicitly. The catalog has no repository without topics (17 have only one), but with a single topic the TF-IDF and SVD signals weaken and PCA similarity and popularity dominate.
- The hybrid weights were set by judgement, not tuned.

12. Graph Analysis and Centrality

In the previous stages repositories were rows or vectors. Here they become the nodes of a network whose connections reveal thematic structure that popularity cannot capture. The question: *which repositories are the most influential in the ecosystem beyond their number of stars?*

12.1. Graph definition

- **Nodes:** repositories. After filtering the graph has 1,938 nodes.
- **Edges:** two repositories are connected if they share at least 4 topics declared by their authors. An edge means thematic co-occurrence; it does not mean dependency, functional equivalence or that one inspired the other.
- **Weight:** number of shared topics. **Undirected:** co-occurrence is symmetric.
- **Threshold:** weights between 2 and 8 were tested. With ≥ 2 the graph has 362,471 edges and density 0.082: generic topics such as `python` connect everything with everything. With ≥ 4 it has 24,839 edges and density 0.013, a balance between connectivity and thematic meaning. This is a modelling decision that affects every later result.

12.2. Construction and structure

The graph is built reproducibly from `repos_clean.csv` in notebook 05: load and parse topics, count shared topics for every pair, create edges with weight ≥ 4, build the NetworkX graph, remove isolated nodes (repositories that share 4 or more topics with no other repository: 1,096) and verify nodes, edges, density and self-loops.

Metric	Value
Nodes after filtering	1,938
Edges	24,839
Density	0.0132
Self-loops	0
Isolated nodes removed	1,096
Connected components	34
Main component	1,861 nodes (96% of the connected repositories), 24,787 edges

The other 33 components have between 3 and 5 nodes: very specific groups that do not connect to the main ecosystem. All centralities below are computed on the 1,861-node main

component.

12.3. Degree and weighted degree

Repository	Degree	Weighted degree	Stars
d2l-ai/d2l-en	280	1,331	28,729
rasbt/LLMs-from-scratch	289	1,327	91,729
bharathgs/Awesome-pytorch-list	277	1,269	16,486
explosion/thinc	267	1,214	2,891
huggingface/datasets	265	1,198	21,469
lukasmasuch/best-of-ml-python	253	1,195	23,453
tangyudi/Ai-Learn	249	1,180	12,946
tensorzero/tensorzero	250	1,176	11,289

12.4. PageRank

PageRank (damping 0.85) measures recursive importance: a repository is influential if influential repositories are connected to it. It is the stationary distribution of a random walker that follows an edge 85% of the time and jumps to a random repository 15% of the time. A value of 0.00507 means that out of 10,000 walkers about 50 would be on `rasbt/LLMs-from-scratch`.

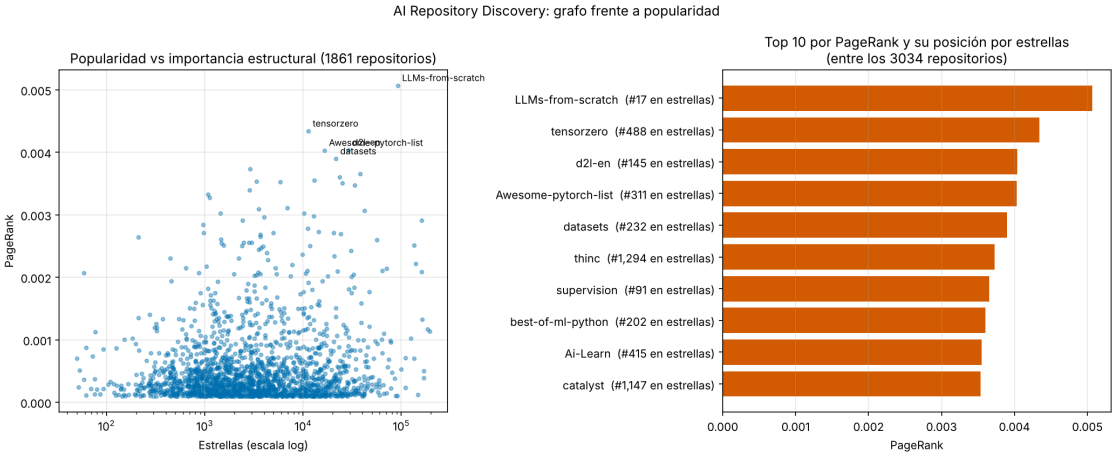
#	Repository	PageRank	Stars	Rank by stars (of 3,034)
1	rasbt/LLMs-from-scratch	0.00507	91,729	17
2	tensorzero/tensorzero	0.00434	11,289	488
3	d2l-ai/d2l-en	0.00404	28,729	145
4	bharathgs/Awesome-pytorch-list	0.00403	16,486	311
5	huggingface/datasets	0.00390	21,469	232
6	explosion/thinc	0.00373	2,891	1,294
7	roboflow/supervision	0.00365	38,259	91
8	lukasmasuch/best-of-ml-python	0.00360	23,453	202
9	tangyudi/Ai-Learn	0.00355	12,946	415
10	catalyst-team/catalyst	0.00354	3,376	1,147

12.5. Betweenness

Betweenness counts how many shortest paths between other pairs of nodes pass through a repository; a high value means a bridge between regions of the graph. Weights were converted to distances (1/weight) before the computation because NetworkX treats weights as distances.

Repository	Betweenness	Stars
rasbt/LLMs-from-scratch	0.112	91,729
linto-ai/whisper-timestamped	0.049	2,807
tensorzero/tensorzero	0.047	11,289
rohitg00/ai-engineering-from-scratch	0.046	5,834
deepset-ai/haystack	0.040	25,024

12.6. Graph ranking versus popularity



Repository	PageRank rank	Rank by stars	Stars
rasbt/LLMs-from-scratch	1	17	91,729
tensorzero/tensorzero	2	488	11,289
explosion/thinc	6	1,294	2,891
catalyst-team/catalyst	10	1,147	3,376
ayoolaolafenwa/PixelLib	15	2,132	1,072
tensorflow/tensorflow	214	1	194,936

- PageRank and stars are almost unrelated: the Spearman correlation across the 1,861 nodes is 0.06. The graph surfaces repositories that popularity hides.

- `explosion/thinc` has 2,891 stars (position 1,294 of 3,034 by popularity) but is 6th by PageRank; `PixelLib` has 1,072 stars (position 2,132) and is 15th.
- `tensorflow/tensorflow` is the most starred repository (194,936) yet only 214th in PageRank: its weighted degree is 385 against more than 1,300 for the top ones, because its topics are specific and shared with few other repositories.

> **Correction.** An earlier version compared PageRank rank with a "rank by stars" calculated only among the 200 repositories with the highest weighted degree (for example 127 for thinc and 180 for PixelLib), and computed betweenness on that 200-node subgraph. Both are now computed on the full catalog and the full main component, which makes the contrast with popularity even larger. The PageRank values and their top-15 order are unchanged.

12.7. Interpretation

- An edge is thematic similarity declared by the authors, not code similarity, functionality or dependency.
- A high PageRank marks a thematic hub that sits on many random-walk paths. It does not mean better code, more usefulness or more maintenance.
- A high betweenness marks a bridge between thematic areas; for instance `rohitg00/ai-engineering-from-scratch` (5,834 stars) has the fourth-highest betweenness.

12.8. Validity checks

- **Symmetry:** the adjacency matrix A (built on the 200 repositories with the highest weighted degree, for readability) satisfies $A = A^T$ (`np.allclose`), so the graph is correctly undirected.
- **Row-stochastic transition matrix:** all rows of P sum to 1.000000, so P is a valid random-walk transition matrix.
- **Self-loops:** 0.
- **Threshold sensitivity:** see the table.

Threshold	Edges	Nodes	Components	Density
2	362,471	2,969	2	0.082
3	94,055	2,582	12	0.028
4	**24,839**	**1,938**	**34**	**0.013**
5	6,739	1,263	68	0.008
6	1,973	770	77	0.007
8	267	265	69	0.008

Threshold 4 keeps a large main component with meaningful connections. Lower thresholds connect repositories through generic topics; higher ones fragment the graph into many small components.

12.9. Limitations

- Results depend entirely on the topics declared by the authors, a subjective and non-standardized signal; repositories without topics or with generic topics are under-represented.
- The threshold ≥ 4 is a modelling decision; another threshold produces different rankings.
- The 1,096 isolated repositories are outside the structural analysis.

13. Conclusions and Limitations

- The dataset (3,034 AI repositories extracted from the GitHub API) combines text, numerical, temporal and relational information and supports every stage of the project.
- Clustering gives five interpretable profiles, but two are tiny (56 and 5 repositories) because heavy-tailed variables enter PCA unscaled; a log scale fixes the balance.
- Text content is what makes the recommender work. SVD's apparent victory is circular, and the hybrid's extra signals favour popular and maintained repositories rather than topical precision.
- The graph shows that structural influence and popularity are nearly independent (Spearman 0.06).
- Limitations: relevance is a topic-overlap proxy; the snapshot is a single point in time (30 April 2026); the hybrid weights are not tuned; 48% of the repositories had no push for over a year.
- Future work: enrich the data with Papers With Code and Hugging Face, tune the weights, and evaluate with real user feedback.

14. Bibliography

GitHub Staff. *Octoverse 2024: AI leads Python to top language as the number of global developers surges*. GitHub Blog, 2024.